they also do not look like they are computing the formula as described. On closer inspection, we see that the network has actually learned something clever: it has used a different representation of the same logical function, specifically:

$$\neg \left(\begin{array}{r} (\neg x_3 \wedge \neg x_{11}) \ \vee \ (\neg x_9 \wedge \neg x_{12}) \ \vee \ (\neg x_2 \wedge \neg x_8) \ \vee \ (\neg x_0 \wedge \neg x_{10}) \vee \\ (\neg x_{13} \wedge \neg x_{14}) \ \vee \ (\neg x_4 \wedge \neg x_7) \ \vee \ (\neg x_1 \wedge \neg x_5) \ \vee \ (\neg x_6 \wedge \neg x_{15}) \end{array} \right)$$

This can be seen to be equivalent to our original presentation of the formula by applying De Morgan's Rule at two different levels: once to the individual OR's of the clauses, and then again to the overall AND of the clauses. Of course, the network has no actual knowledge of De Morgan's Rule - it is simply given a subset of the truth table for the formula, and it learns some representation of that truth table. It is interesting to note that based on this example, the network does seem to have a propensity to learn DNF formulas instead of CNF formulas.

In a bit more detail, we see that the network is actually learning this formula as follows. The outer negation is realized through the utilization of the negative layer 2 weights: if a positive signal comes through the channel codes, then that is converted into a significant negative value by these negative weights. The neuron at the second layer takes the OR of these negative weights: if any one of them evaluates to a (large enough) negative value, then the entire system evaluates to a negative value. If none of them are negative, then the small positive weights on the postive layer 2 weight rows are enough to make the whole system positive. Each of the clauses is computed through a distinct feature channel code, and relies on the fact that $(\neg x_1 \wedge \neg x_2)$ can be computed in soft lagic as $\text{ReLU}(b - x_1 - x_2)$ where $b = 1$ in the binary case. Due to the negative values of the variables, the codes take on negative values (and so are depicted in red in this figure). We note that the layer 1 bias is usually positive (corresponding to the +1 in that formula), but it is not usually as large as that formula dictates, which is analogous to what we see with DNF formulas, except with the opposite sign. Here though, some of the rows are negative, which is not analogous to the DNF formulas covered above, where we saw consistently negative bias in the positive weight rows. However, this is compensated for by positive values that appear elsewhere in the row. Due to the way the variables were set for the inputs, there is a high likelihood that every row will have an additional positive contribution, to make up for the slightly smaller bias.

5.2 Code Accuracy in a One Dimensional Vision Problem

We next turn our attention to a problem that extends beyond Boolean formulas, into a pattern matching problem that could be viewed as a one dimensional vision problem. We here consider the *consecutive four* problem: given an input string of binary values, are there 4 ones in a row? Thus the individual input variables are again binary. We consider this an extremely simple vision problem, in the sense that it deals with pattern matching with locality. With that locality in mind, we we will train and test our network with a restricted set of inputs to this problem: all inputs have exactly six ones in the string, and they must all be close to each other - within a region of eight consecutive bits of the input string. We want to see if neural networks trained to solve this problem have properties that are consistent with what we see with pure Boolean problems.

We point out that, like many pattern matching problems, this problem can be solved with a Boolean formula: the OR of all four ways ANDs of four consecutive bits. And in fact, even though the training process is not given this representation (only the ordered sets of inputs and their labels),

using the Combinatorial Interpretability approach, we see that the networks we train to solve this problem do in fact use exactly that formula.

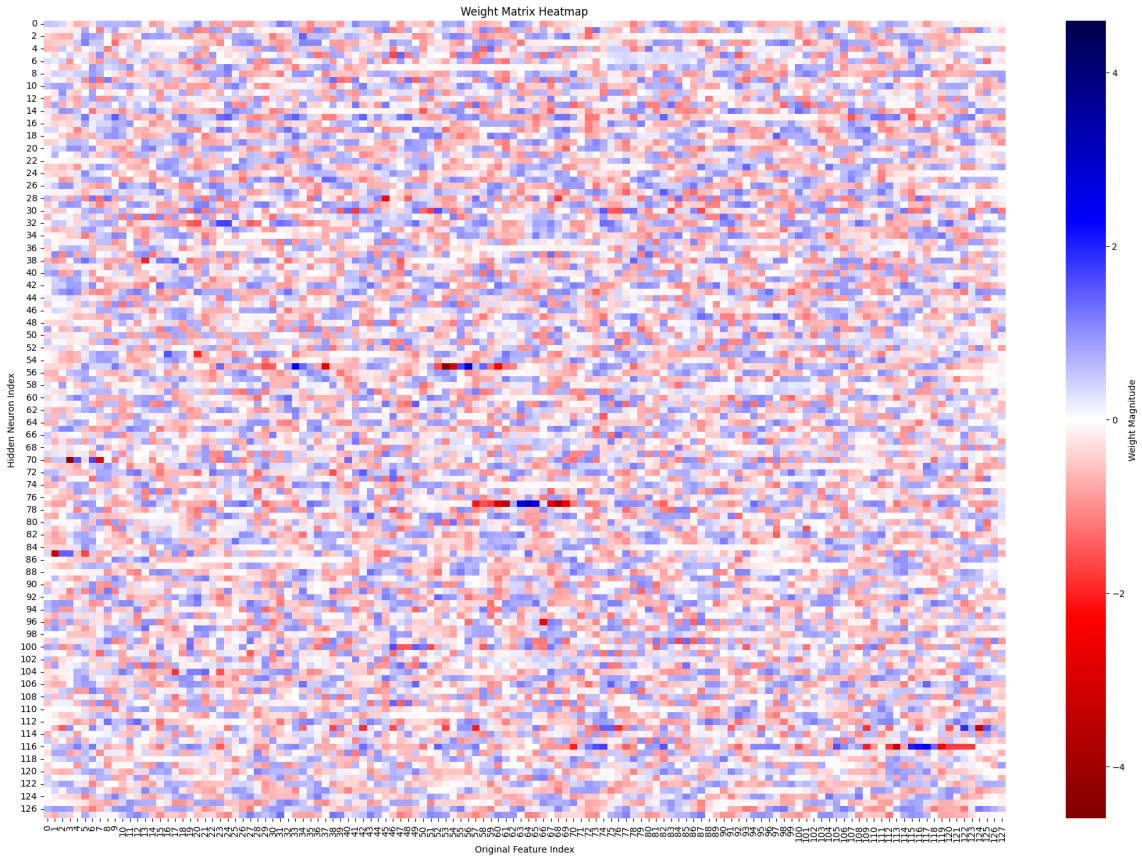

Figure 16: Layer 1 heat map for the connect four problem. The x-axis corresponds to the pixels of the one dimensional image we the network receives as input.

Our setup is as follows. We use a network with 128 input variables, 128 hidden neurons in the single hidden layer, and a single output neuron. We generate each training and test data input as follows:

- Flip a fair coin for each input to determine if it will be positive or negative.
- In either case, choose a sequence of eight consecutive variables uniformly at random.
- For a positive input, out of the set of eight chosen variables, pick a consecutive set of four variables uniformly at random. Also choose two other variables from the eight uniformly at random. These six variables are set to 1, and all others are set to 0.
- For a negative input, randomly pick a consecutive set of four variables out of the eight, but only set three of them to 1. Then pick another three of the eight variables uniformly at

random, and set those variables to 1 as well. Then test if the result has four consecutive variables set to 1. If it does, repeat until a negative input is generated, or 20 attempts have failed, in which case use the string of all zeros.

We generated 10,000 training inputs, and trained for 20 epochs, after which the network had 0 loss. Accuracy on the test set was 99%. The resulting weight matrix, shown in Figure 16, at first glance, might not seem to exhibit channel coding.

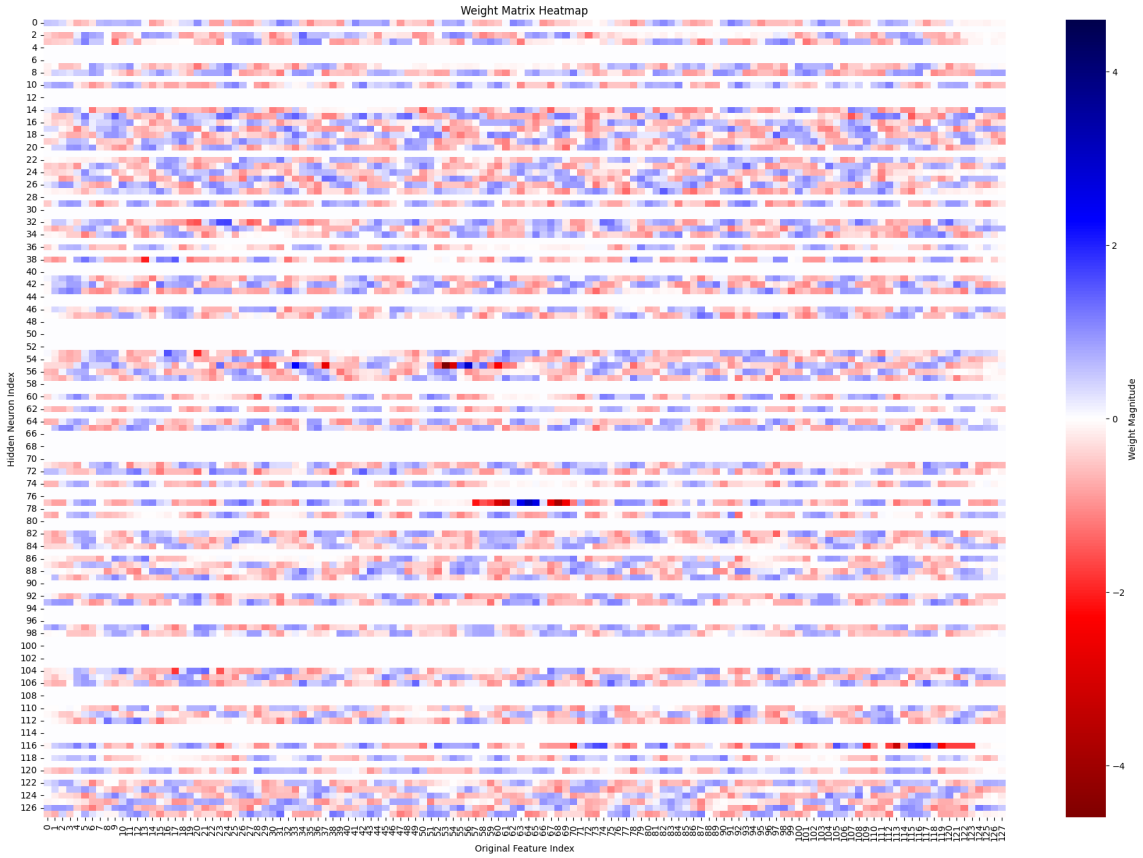

Figure 17: Layer 1 for connect four: positive witnesses.

However, things become quite a bit clearer when we separate the rows into positive witnesses and negative witnesses, where (as before) a row is considered a positive witness if its Layer 2 weight is positive, and a negative witness if its Layer 2 weight is negative. The positive witness only heat map is depicted in Figure 17. This is a subset of the rows shown in Figure 16; no other changes have been made.

We can see visually that the positive witness rows do in fact demonstrate considerable feature channel coding. And in fact, we see that most of the rows of the (positive only) matrix consist of alternating sequences of four consecutive positive values, followed by a number of negative values. There are some sequences of ones that are longer than four, but most of them have length exactly